

DEVELOPER ONBOARDING GUIDE

EDGE.

Standard Developer Procedures

Everything you need to know to start contributing to the EDGE trading toolkit — from first-time setup through daily development workflows, Git best practices, and session handoff procedures.

PROJECT

EDGE Trading Toolkit

REPOSITORY

github.com/ianmaser/trading-toolkit

VERSION

April 2026

CONTENTS

1	First Time Setup	
2	Starting Every Session	
3	Working With Claude Code	
4	Saving Your Work	
5	Ending Every Session	
6	Code Quality Checklist	
7	After Completing a Phase	
8	Weekly Audit	
9	Git Troubleshooting	
10	File Map	



THE GOLDEN RULE

**Always pull before you start.
Always push when you stop.**

1

First Time Setup

ONCE ONLY

If you have never worked on this project before on your computer, follow every step below in order. You only do this once — skip to Section 2 after that.

What is a terminal?

The terminal is a text-based window where you type commands to control your computer. Think of it as texting your computer instructions. Everything is case-sensitive.



Mac

Press **Cmd + Space**, type **Terminal**, press Enter.



Windows

Press the **Windows key**, type **PowerShell**, press Enter.

Install prerequisites

Install these in order. After each one, close and reopen your terminal before moving on.

1

Node.js — the engine that runs the app

Download from nodejs.org — choose the LTS version. Verify: type `node -v` in your terminal. You should see something like `v20.18.0`.

2

Git — the tool that saves and shares code

Download from git-scm.com. Verify: type `git -v`. You should see `git version 2.x.x`.

3

VS Code — the code editor

Download from code.visualstudio.com. This is where you read and review code.

4

Claude Code — the AI that builds with you

Open VS Code → Extensions panel (four-square icon) → search **Claude Code** → Install. Or run `npm install -g @anthropic-ai/claude-code` in your terminal.

Recommended VS Code extensions

Extension	What it does
ESLint	Highlights code errors as you type

Extension	What it does
Prettier	Automatically formats code so it stays clean
Tailwind CSS IntelliSense	Autocompletes CSS class names
GitLens	Shows who wrote each line and when

Clone the project from GitHub

"Cloning" downloads the full project to your computer. Run these two commands, one at a time:

```
terminal

# Download the project
$ git clone https://github.com/ianmaser/trading-toolkit.git

# Navigate into the project folder
$ cd trading-toolkit

# Open it in VS Code
$ code .
```

Install dependencies

This downloads all the libraries the project needs. It may take a minute — the scrolling text is normal.

```
terminal

$ npm install
```

Set up your environment variables

The project needs API keys (secret passwords for external services) to run. These are **never stored in Git** for security reasons.



Important — never skip this step

Ask your partner to share the `.env.local` file with you directly (iMessage, email, etc.). Place it in the root of the project folder — the same level as `package.json`. Never share this file publicly. Never commit it to Git.

Confirm everything works

```
terminal

$ npx tsc --noEmit
```



No output = no errors

If you see nothing after running this command, the codebase is clean and you're ready to go. If you see red error messages, stop and message your partner before doing anything else.

2

Starting Every Session

EVERY TIME

This is your pre-flight checklist. It takes 2 minutes and prevents hours of headaches. Do not skip steps — they exist for good reasons.

1 Open your terminal in the project folder

In VS Code: top menu → **Terminal** → **New Terminal**. This opens a terminal already pointed at the project — no navigating needed.

2 Get the latest code from GitHub

Your partner may have made changes since you last worked. Always pull first.

```
terminal  
$ git pull origin main
```



If you see "CONFLICT" in the output

Stop immediately. A conflict means two people edited the same file at the same time. Do not touch anything. Message your partner — resolve it together.

3 Install any new dependencies

Safe to run every time — does nothing if nothing changed.

```
terminal  
$ npm install
```

4 Confirm the codebase is clean

No output = no errors. If there are errors, do not start building — fix them first or ask your partner.

```
terminal  
$ npx tsc --noEmit
```

5 Read the session notes

Open the `sessions/` folder and read the most recent file. It tells you exactly where the last session left off and what to do next.

6 Read the build plan for today's work

Open `PLAN.MD` and find the prompt for the feature you're building. Read it fully before opening Claude Code. If the feature touches API routes, caching, auth, or BULL-E, also skim the relevant section of `ARCHITECTURE.md`.

3 Working With Claude Code

Claude Code is an AI assistant that writes code alongside you. It reads the project files, understands the architecture, and builds features based on your instructions. It does **not** remember previous sessions — you must give it context every time.



How to start a feature — use this prompt every time

"Read CLAUDE.md and PLAN.MD. We're building *[feature name from PLAN.MD]*. Here's exactly what it needs to do: *[paste the full prompt text from PLAN.MD]*"

1

One prompt at a time

Never ask Claude to build two features in one message. You will always get worse results when you combine prompts.



When something breaks

Paste the **full error** and the **full file**. Say: "Don't rewrite the whole file. Find the specific problem and fix only that."



Ask Claude to write session notes

Say: "Record everything we implemented today in a session note titled *04-16-2026-phase-01.md*"



Claude has no memory

Never skip the context prompt at the start. Starting without it is like asking someone to continue a conversation they weren't in.

4

Saving Your Work

AFTER EVERY PROMPT

Once a prompt is done and `npx tsc --noEmit` shows no errors, save your work to GitHub using these four steps in order.

1

Check what changed

Review every file listed. Make sure you recognise everything before saving.

 terminal

```
$ git status
```

2

Stage your changes

Tells Git which files to include in this save. `.env.local` is automatically excluded — you will never accidentally commit your API keys.

 terminal

```
$ git add .
```

3

Commit your changes

A commit is a named save point. Use this format every time: `feat: Phase {N} Prompt {N} – {short description}`

 terminal

```
$ git commit -m "feat: Phase 1 Prompt 5 – market data service"
```

4

Push to GitHub

Uploads your commit so your partner can see it. You'll see output ending in `main → main` when it works.

 terminal

```
$ git push origin main
```

5

Ending Every Session

EVERY TIME

Claude does not remember your last session

Every time you open Claude Code, it starts with zero memory of what you built before. The only way it knows what happened is if you tell it — and the only way your partner knows is if you write it down. Write a session note every time you stop, even if you only worked for 30 minutes and didn't finish anything.

1

Ask Claude to write the session note

Tell Claude: *"Record everything we implemented today in a session note titled MM-DD-YYYY-short-description.md"*

Examples: `04-16-2026-phase-01-market-data.md` · `04-17-2026-backtest-bugfix.md`



Session note template

```
sessions/MM-DD-YYYY-title.md

# Session – MM-DD-YYYY – [Title]

## Who worked on this
[Your name]

## What was built
- Prompt X: [what you built and where the files are]

## What is NOT finished yet
- [e.g. "Started Prompt 6 but Redis caching not wired up yet"]

## Left off at
- [Exactly what the next person should do first]

## Decisions made
- [Choices not in PLAN.MD or ARCHITECTURE.md]

## Open questions
- [Anything unresolved the other person needs to decide]
```

2**Commit and push the session note**

Never leave without pushing. Your partner's next session depends on it.

 terminal

```
$ git add sessions/  
$ git commit -m "docs: session notes MM-DD-YYYY"  
$ git push origin main
```

6

Code Quality Checklist

BEFORE EVERY COMMIT

Run through this before marking any prompt as done. Every item must be checked before you commit.

- ☐ `npx tsc --noEmit` — zero errors, no output
- ☐ No `any` types in any new code
- ☐ No hardcoded URLs, API keys, or secrets — everything is in `.env.local`
- ☐ TypeScript only — no `.js` files created
- ☐ All new UI components have a loading skeleton and an error state
- ☐ All buttons and interactive elements have an `aria-label`
- ☐ All API inputs are validated with Zod
- ☐ Mobile layout looks correct at 375px width

7

After Completing a Phase

1

Ask Claude to write tests

"Write tests for everything we just built — happy path, empty states, and error conditions."

2

Manually test the user journey

Click through the full flow affected by the phase. Confirm it works end to end in your browser.

3

Update the phase status in CLAUDE.md

Open `CLAUDE.md` and change the phase from  to  in the feature map table.

4

Commit, push, write your session note

Follow sections 4 and 5 as normal. Then move to Section 8 if it's Friday.

Copy and paste this exactly into Claude Code every Friday:



Friday audit prompt

"Review all the files we've built this week. Look for: any hardcoded values that should be env vars, any missing error handling, any component that's missing a loading state, any Supabase query missing RLS, any TypeScript any types. List everything you find and fix them one by one."

Git Troubleshooting



"Push rejected" or "failed to push"

Your partner pushed something after your last pull. Fix it by pulling first, then pushing again.



terminal

```
$ git pull origin main  
$ git push origin main
```



"CONFLICT" when pulling

Two people edited the same file. Do not touch anything. Message your partner — resolve it together.



"I accidentally committed .env.local"

Message your partner immediately. This is a security issue that needs to be handled together before the commit reaches GitHub.



"I don't know what state my code is in"

Run `git status` to see every file that has changed. Run `git log --oneline -10` to see the last 10 commits and their descriptions.



"I want to undo my last commit (before pushing)"

Run `git reset --soft HEAD~1` — this un-commits your last save but keeps all your changes. Only use this **before** pushing. If you've already pushed, message your partner first.

10 File Map — Where Things Live

What you're looking for	Where it is
What to build next	<code>PLAN.MD</code>
How the app is wired together	<code>ARCHITECTURE.md</code>
Rules, stack, current phase	<code>CLAUDE.md</code>
Database tables and columns	<code>supabase/migrations/001_initial.sql</code>
Shared TypeScript types	<code>types/</code>
Utility functions (auth, cache, rate limiting)	<code>lib/</code>
React hooks (data fetching)	<code>hooks/</code>
API services (market data, signals, etc.)	<code>services/</code>
Page components	<code>app/dashboard/</code>
Reusable UI components	<code>components/features/</code> and <code>components/ui/</code>
Session handoff notes	<code>sessions/</code>
Things flagged to revisit	<code>TODO/revisit-later.md</code>
API keys (never in Git)	<code>.env.local</code> — shared privately with partner



REMEMBER

Session notes are your handoff to your partner — and to Claude. Always write one before you log off.